Taller de Programación Orientada a Objetos

Modelos y Paradigmas de Programación Escuela de Ingeniería de Sistemas y Computación - Universidad del Valle

Estudiante: Raúl Andrés Alzate Gómez

Ejercicio 1: Implementando herencia múltiple en general (NuevaHeredarDe)

1.1 Contexto: El sistema de objetos simplificado (Sección 7.6 CTM)

En la sección 7.6 del libro *Concepts, Techniques, and Models of Computer Programming* se presenta una implementación simplificada de un sistema de objetos. En este modelo:

- **Una clase** se representa como un registro `c(metodos:M atrbs:A)` donde:
  - `metodos` es un registro que mapea nombres de métodos a procedimientos
  - `atrbs` es una lista de átomos que representan nombres de atributos
- **Un objeto** es un procedimiento que recibe un mensaje y despacha al método correspondiente
- Las clases se **empaquetan** con `Envolver`/`Desenvolver` (TAD declarativo seguro) para garantizar encapsulamiento

Este modelo demuestra que los objetos pueden implementarse con dos conceptos fundamentales: - **Estado explícito**: cada atributo se implementa como una celda (`NewCell`), y el estado del objeto es un registro de celdas compartidas por todos sus métodos - **Procedimiento como valor**: cada método es un procedimiento `proc {Metodo Msg Estado Self}` que recibe el mensaje, el estado compartido y la referencia al propio objeto (self)

La función `Nuevo` (equivalente a `New`) crea un objeto a partir de una clase:

```
fun {Nuevo ClaseEmp MetodoInicial}
   Estado Obj Clase={Desenvolver ClaseEmp}
in
   Estado={MakeRecord s Clase.atrbs}
   {Record.forAll Estado proc {$ A} {NewCell _ A} end}
   proc {Obj M}
      {Clase.metodos.{Label M} M Estado Obj}
   end
   {Obj MetodoInicial}
   Obj
end
```

El despacho de métodos funciona así: cuando se invoca `{Obj msg(...)}`, el procedimiento `Obj` extrae la etiqueta del mensaje con `{Label M}`, busca el método correspondiente en `Clase.metodos`, y lo invoca pasándole el mensaje, el estado y el propio objeto.

1.2 La función original HeredarDe

La función `HeredarDe` del libro combina una clase base con exactamente dos superclases:

```
fun {HeredarDe C1 C2 C3}
   c(metodos:M1 atrbs:A1)={Desenvolver C1}
   c(metodos:M2 atrbs:A2)={Desenvolver C2}
   c(metodos:M3 atrbs:A3)={Desenvolver C3}
   MetEnConfl={Minus {Inter {Arity M2} {Arity M3}} {Arity M1}}
   AtrEnConfl={Minus {Inter A2 A3} A1}
in
   if MetEnConfl\=nil then
      raise herenciaIlegal(metEnConfl:MetEnConfl) end
   end
   if AtrEnConfl\=nil then
      raise herenciaIlegal(atrEnConfl:AtrEnConfl) end
   end
   {Envolver c(metodos:{Adjoin {Adjoin M2 M3} M1}
               atrbs:{Union {Union A2 A3} A1})}
end
```

La semántica es: - **Prioridad de métodos**: C1 (clase base) > C3 > C2. La función `Adjoin` da

prioridad al segundo argumento, así que `{Adjoin {Adjoin M2 M3} M1}` resulta en M1 sobreescribiendo a M3, que sobreescribe a M2. - **Atributos**: unión de conjuntos de las tres clases (no hay prioridad, solo presencia). - **Validación**: detecta conflictos cuando dos superclases definen el mismo método o atributo y la clase base no lo resuelve.

1.3 Generalización: NuevaHeredarDe

La función `NuevaHeredarDe` generaliza `HeredarDe` para recibir una lista arbitraria de superclases:

```
fun {NuevaHeredarDe ClaseEmp SuperClasesEmp}
```

Estrategia de implementación:

1. **Desenvolver todas las clases**: Se desempaqueta la clase base y cada superclase de la lista usando `Desenvolver`.

2. **Validación por pares**: Se verifican todas las combinaciones de pares de superclases (Si, Sj) donde i < j. Para cada par se calcula:

   - Conflictos de métodos: `{Minus {Inter {Arity Si.metodos} {Arity Sj.metodos}} MA_Base}`
   - Conflictos de atributos: `{Minus {Inter Si.atrbs Sj.atrbs} A_Base}`

   Si alguno es no vacío, se lanza la excepción `herenciaIlegal`.

   La validación se implementa con dos procedimientos recursivos:

   - `ValidarPares`: recorre la lista de superclases
   - `CheckPairwise`: para cada superclase, la compara con todas las siguientes

3. **Combinación de métodos** (`MergeMethods`): Se hace un fold recursivo con `Adjoin` sobre la lista de superclases. Para `[S1 S2 ... Sn]`:

   - `{Adjoin S1.metodos {Adjoin S2.metodos {... Sn.metodos}}}`
   - Prioridad entre superclases: Sn > Sn-1 > … > S1 (la última tiene mayor prioridad)
   - Luego `{Adjoin resultado Clase.metodos}` asegura que la clase base siempre tiene máxima prioridad

4. **Combinación de atributos** (`MergeAttrs`): Unión de conjuntos de todas las listas de atributos, seguida de unión con los atributos de la clase base.

5. **Empaquetado**: El resultado se envuelve con `Envolver` para mantener el encapsulamiento.

1.4 Verificación con las pruebas

PruebaHerenciaExito - Herencia exitosa con 2 superclases: - Contador (base): métodos `{browse, inic, inc}`, atributos `[val]` - Disminuir: método `{dism}`, atributos `nil` - Restaurar: método `{restaurar}`, atributos `nil` - No hay intersección de métodos entre Disminuir y Restaurar, por lo tanto no hay conflictos - Resultado: clase con métodos `{browse, inic, inc, dism, restaurar}` y atributo `[val]`

PruebaHerenciaFalla - Herencia con conflicto (3 superclases): - Contador (base): métodos `{browse, inic, inc}`, atributos `[val]` - Disminuir: método `{dism}` - DisminuirMul: métodos `{dism, dismmul}` - Restaurar: método `{restaurar}` - Intersección de métodos entre Disminuir y DisminuirMul: `{dism}` - `{Minus [dism] [browse inic inc]}` = `[dism]` (no está en la clase base) - Se lanza `herenciaIlegal(metEnConfl:[dism])` correctamente

---

Ejercicio 2: Objetos evaluadores - Polimorfismo (Fórmulas)

2.1 Diseño de clases

Se implementan 6 clases usando el sistema de clases estándar de Mozart/Oz para representar y evaluar fórmulas matemáticas. Cada subexpresión es un objeto:

Clase	Atributos	Métodos	Descripción
Variable	`val`	`inic(V)`, `asg(V)`, `evaluar(X)`	Variable con valor asignable
Constante	`val`	`inic(V)`, `evaluar(X)`	Valor numérico constante
Suma	`e1, e2`	`inic(E1 E2)`, `evaluar(X)`	Suma de dos expresiones
Diferencia	`e1, e2`	`inic(E1 E2)`, `evaluar(X)`	Resta de dos expresiones
Producto	`e1, e2`	`inic(E1 E2)`, `evaluar(X)`	Multiplicación de dos expresiones
Potencia	`base, exp`	`inic(B E)`, `evaluar(X)`	Base (expresión) elevada a exponente (entero)

2.2 Polimorfismo

El método `evaluar(X)` es **polimorfo**: se puede invocar sobre cualquier objeto sin importar su clase concreta. Cada clase implementa `evaluar` de manera diferente:

- **Variable**: retorna el valor almacenado en su celda `@val`
- **Constante**: retorna su valor constante `@val`
- **Suma**: evalúa recursivamente ambas subexpresiones y retorna su suma
- **Diferencia**: evalúa ambas subexpresiones y retorna su diferencia
- **Producto**: evalúa ambas subexpresiones y retorna su producto
- **Potencia**: evalúa la base recursivamente y la eleva al exponente entero

Esto permite construir árboles de expresiones donde cada nodo es un objeto. La evaluación se propaga recursivamente desde la raíz hasta las hojas (variables y constantes), sin necesidad de conocer la estructura interna de cada subexpresión.

2.3 Estado y reasignación

La clase `Variable` demuestra el uso de **estado explícito**: el atributo `val` es una celda que se puede reasignar con el método `asg(V)`. Esto permite evaluar la misma fórmula múltiples veces con diferentes valores, sin reconstruir el árbol de expresiones.

2.4 Ejemplo: Fórmula 3x^2 - xy + y^3

La fórmula se construye como un árbol de objetos:

```
          Suma
         /    \
   Diferencia  Potencia(y,3)
    /      \
Producto  Producto
  /  \      /  \
 3  Pot    x    y
    (x,2)
```

Evaluación con x=7, y=23: - 3 * 7^2 = 3 * 49 = 147 - 7 * 23 = 161 - 147 - 161 = -14 - 23^3 = 12167 - -14 + 12167 = **12153**

Evaluación con x=5, y=8: - 3 * 5^2 = 3 * 25 = 75 - 5 * 8 = 40 - 75 - 40 = 35 - 8^3 = 512 - 35 + 512 = **547**

Conceptos clave demostrados

1. **Objetos como registros con procedimientos embebidos compartiendo estado** : En el sistema simplificado (Ejercicio 1), cada objeto es un procedimiento que contiene un registro de celdas (estado) compartido por todos sus métodos. Los métodos son procedimientos almacenados en la clase que acceden al estado mediante `Access` y `Assign`.

2. **Herencia**: La función `NuevaHeredarDe` combina clases fusionando sus tablas de métodos (con `Adjoin`) y uniendo sus listas de atributos (con `Union`), respetando una jerarquía de prioridad donde la clase base siempre prevalece.

3. **Self y despacho de métodos** : En `Nuevo`, el parámetro `Self` (alias `Obj`) se pasa a cada método, permitiendo que un objeto se invoque a sí mismo. Esto es esencial para que la

herencia funcione correctamente: un método heredado puede invocar otros métodos del objeto, incluyendo los sobreescritos.

4. **Polimorfismo**: En el Ejercicio 2, el método `evaluar(X)` se implementa en todas las clases, permitiendo tratar uniformemente cualquier subexpresión independientemente de su tipo concreto. El despacho del método correcto ocurre automáticamente según la clase del objeto receptor.